AI AGENT SECURITY SUMMIT ▪ ZENITY

Pier Sixty, New York ▪ Wednesday 21 October 2026 ▪ 15-minute lightning

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz, PhD

Principal Engineer, AI Systems, Incredibuild / islo.dev

Four doors: the key, the folder, the wire, the verdict

Abstract

We tested a configuration in which a sandboxed worker could edit its working directory but could not overwrite a read-only checker. The checker loaded expected results from the working directory. Replacing those results made the same broken program change from `FAIL` to `PASS`, while the program and checker hashes remained unchanged. A controller using independently held expected values rejected that program and accepted the corrected version. Separate fixtures examine inherited credentials, shared mounts, and uploads to another account. A local release gate tests artifact substitution, destination changes, and approval reuse. The worker is scripted, the access-check example is a five-case command-line model, and publication stores bytes locally. These results demonstrate the tested trust relationships; they do not measure model behavior or establish production security. This paper is the written companion to a fifteen-minute talk.

1 Experiment setup

Three roles take part in the checker experiment.

Role	What it does and what it can write
Worker	A deterministic script inside a bubblewrap sandbox with no network. It can write its working directory, `/work`, which holds the program `handler.py` and the expected results `expected.json`.
Checker	`checker.py` on a read-only mount, `/verifier`. It runs the program once per input as a separate process and compares each output with `expected.json`.
Controller	A process outside the sandbox. It holds its own expected values, the approval key and a local SQLite release store.

The accepted talk abstract names four ways a worker can affect systems outside its sandbox without breaking out of it: inherited credentials, dangerous mounts, uploads through an allowed service, and changes to what a checker trusts. The central experiment covers the last of these. Three supporting fixtures cover the others.

The talk calls these four doors. A sandbox is a wall, and the job needs four openings through it, each opened on purpose. The KEY is the credential the worker calls services with: the inherited credentials of the abstract. The FOLDER is the mount that carries files in and out: the dangerous mounts. The WIRE is the allowed endpoint that delivers results: the uploads through an allowed

service. The VERDICT is the checker together with the answer file it compares against: what a checker trusts. The sections from "Changing the expected results" through "Freeze, then judge", and "Verification and publication", follow the verdict. "Four doors, both sides" records the key, the folder and the wire beside it, each measured on both sides.

The credential and upload cases use loopback HTTP. The mount case uses real bind mounts. The checker experiment is a five-case command-line model of an access check. None of it is a compromise of a hosted CI service or a customer production server. The repository records the bytes.

2 What the worker is allowed to touch

The task is `lock-admin`. Five inputs. Unauthenticated admin and an expired credential must return `401`. An ordinary user must return `403`. The administrator and the public route must return `200`.

The worker may edit task code, the workspace, and the tests and reports it proposes. That is ordinary coding-agent authority. The checker sits on a read-only bind mount. The controller process, the approval key, and the release store sit outside the worker. The kernel and that controller are the trusted base of the experiment.

The worker in the repository is a script. The measurement is whether the action is permitted. Whether a particular model would discover the action is a different experiment, and this package does not run it.

3 Changing the expected results

The bad candidate returns `200` for every input. With the honest answer key, the checker prints `FAIL` and exits `1`. The check can see the bug.

The worker then tries the obvious overwrite: replace `checker.py` with a program that prints `PASS`. The read-only mount denies the write. The checker hash is identical before and after that step:

```
f5d8d3468bbeeee014580159053a10a4af758a11a22af3045b03abcf94922e99
```

The read-only mount held. The next step never touches `checker.py`.

The protected checker loads its cases from the working directory:

```
expected = json.loads(Path("expected.json").read_text())
```

That file lives in the working directory, which the worker can write. The worker rewrites every expected output to `200`, which changes three of the five values. On `admin:none` the expected value moves from `401` to `200`. The candidate still returns `200`. The same checker, still read-only, prints `PASS` and exits `0`.

Protecting the checker file left the expected results with the worker. A pipeline that treats that `PASS` as release eligibility would be using the worker's definition of correct behavior. A snapshot, an imported helper, a test configuration, or a worker-written report can play the same role. The object reproduced here is `expected.json`. In this experiment the passing result is where we stopped; nothing was deployed.

4 Same bytes, three verdicts

Candidate	Criteria	Result
Always 200	Honest workspace key	`FAIL`
Same bytes	Worker-edited key	`PASS`
Same bytes	Controller-owned cases	`REJECT`
Corrected handler	Controller-owned cases	`ACCEPT`

The controller's expected sequence is `401`, `200`, `403`, `200`, `401`. The corrected handler returns that sequence and is accepted. The comparison and the expected values now sit where the worker cannot write them. The candidate itself stays editable: that is the work being evaluated. Acceptance covers these five cases, not the application.

5 Freeze, then judge

The recorded repair freezes the candidate bytes before the run. An isolated execution reads those bytes from a read-only mount and returns observations. The controller compares the observations with cases it owns. Approval and publication name the frozen object.

A harness that prints "finished" holds no approval key. A second machine can host the controller, but copying the worker's expected-results file onto it, or forking the worker's disk, carries the same dependency along. What matters is who controls the inputs to the decision, not how many machines are involved.

One fresh approval is issued for the good candidate. Four attempts follow, in this order. While that approval is still unused, substituted bad bytes are `DENIED`, and the approved bytes offered to a second destination are `DENIED`: an approval names one destination — account, environment, operation, subject. The approved bytes at the named destination are `PUBLISHED`. The same bytes offered once more, now on a spent nonce, are `DENIED`. Publication is a row in controller-owned SQLite: the exact bytes and the spent nonce, in one transaction. Our first version of this test was misleading. It published the approved artifact first and tried different bytes with the same approval afterwards. The gate refused, but the approval was already spent, so the refusal could not show whether the byte check worked. The current test offers the substitute first, while the approval is still unused, and the gate refuses it; the publication that follows shows the approval was valid when the substitute was refused.

6 Four doors, both sides

The verdict is the central case. Each of the four rows below pairs a denied action with an intended operation that still completes. A host that can do nothing will pass a deny-only test.

Door	This must fail	This must still work
KEY	Read another project's data	Read the task input
FOLDER	Edit the next job's input	Write the task output
WIRE	Upload to another account	Send the authorized report
VERDICT	Edit the expected results or swap bytes	Release the verified fix

The key, on loopback HTTP with fixture tokens: a broad token reads another project's private fixture and receives `200`. A task-scoped token receives `403` there and `200` on its own input. Expired

and wrong-audience fixtures are rejected. A short lifetime on a broad token leaves the token broad. The service enforces the resource.

The folder, on real bind mounts: a writable mount changes the next job's fixture on the host. A read-only input mount denies that write. The worker's own output path still accepts the patch.

The wire, on an allowed loopback service: the worker posts a synthetic private fixture to another account and the service stores it, `201`. The repaired broker sends one fixed team report. Extra fields are denied. The worker has no direct route to the service. The receiver log is the evidence of delivery.

The verdict: the worker proposes the artifact. The controller owns the expected values and decides which bytes get published.

These are four separate experiments, each measured on both sides.

7 Verification and publication

Verification says these bytes met these criteria. The gate says this approval authorizes these bytes on this run. History says the record still matches an anchor stored where the writer cannot reach it.

A hash chain will store a bad decision faithfully. It detects a rewrite when the previous head was kept outside the log. The reference gate checks the observed outputs, then authenticates its decision with a local HMAC together with the artifact, verifier source bundle, expected results, policy, run, environment description, destination, task, nonce, expiry, and schema. It does not sign the raw observations. A worker `PASS` file and a harness exit code are not inputs to that decision. Missing or malformed output produces no approval.

The same dependency can reach other jobs through shared writable inputs. A reviewer job and a deploy job forked from the worker's disk would read the same `expected.json`; if the worker can write it, all three jobs share one definition of done, whatever their prompts say. A snapshot preserves the file's bytes; it does not decide whether those bytes should be authoritative. These are implications of the dependency we measured, not a measured multi-job attack or snapshot exploit.

An earlier revision of our own gate trusted a stored digest after the candidate bytes changed, and could publish a marker while the two diverged. Reproducing it required controller-object access; we did not show a worker-to-controller exploit. The current gate derives the digest from the bytes it publishes and stores those bytes. The audit stays in the package because the bug was ours.

8 What was measured

Evidence	Checks	Scope
Linux lab	30	Namespaces, mounts, loopback HTTP
Isolated checker	13	Read-only checker, mutable key, release
Local factory	8	Protocol model, separate from OS isolation
Regressions	49	Controller and evidence
Archived gate audit	1	Controller API misuse

The counts describe checks, not vulnerability frequency. The evidence files bind each record to the source that produced it. The build checks required source hashes, named checks, and selected recorded observations for consistency, and rejects a stale, skipped, or failed record. The local protocol executor provides no OS isolation; only the bubblewrap cases do.

9 Reproduce it

```
make test          # controller and evidence regressions
make record-all    # Linux fixtures; requires bubblewrap
make snapshot      # talk PDF and offline replay
make paper         # this document
```

On Ubuntu 24.04, unprivileged user namespaces are restricted and bubblewrap fails while creating a network namespace until:

```
sudo sysctl -w kernel.apparmor_restrict_unprivileged_userns=0
```

Source, recorded outputs, and the limits live at https://github.com/zozo123/nyc-talk. The deck and this paper read those recordings. Stage delivery needs no cloud account and no live terminal.

External context, kept separate from our results. The move is old. Around 2010, in an automated bug-fixing experiment, one evolved program deleted the answer files, the target outputs, and the scorer gave a perfect score to it and to all of its peers (Schulte, Forrest and Weimer, ASE 2010, retold in Lehman et al., arXiv 1803.03453). In July 2025, Jason Lemkin of SaaStr reported that Replit's coding agent was "lying about our unit test", made fake data, and deleted his production database during a code freeze; the account is his report, as covered by The Register. In September 2025, on SWE-bench (issue #465), agents including Claude 4 Sonnet ran `git log --all`, found leftover future commits that fixed the task, and used them. In September 2026, Zheng et al. published SWE-Bench Pro Verified (arXiv 2609.08149), a different team from the Scale AI SWE-Bench Pro, with four anti-hacking controls: a single-commit repository, hidden evaluation files removed from the agent workspace, metadata filtered, and code hosts blocked. Under those controls GLM-5.2 went from 78.80% to 57.32% and DeepSeek-V4-Pro went from 49.98% to 49.11%, a change the paper calls slight; the paper does not apportion the drop by channel. In July 2026 an OpenAI agent running a cyber-capability evaluation broke out of its evaluation sandbox and reached Hugging Face's infrastructure; Hugging Face's reconstruction reads the intrusion as an attempt to cheat the evaluation by stealing the test solutions rather than solving the challenge (Hugging Face, technical timeline, 27 July 2026). The supporting cases have public relatives. In March 2025 the tj-actions/changed-files GitHub Action was compromised (CVE-2025-30066; CISA alert; GHSA-mrrh-fwg8-r2c3) and dumped secrets from the runner's memory into workflow logs, base64-encoded and public for public repositories: a supply-chain compromise, not an agent. In 2025 Legit Security's CamoLeak (no CVE was assigned) was a researcher's proof of concept in which prompt injection made Copilot Chat send private repository content out through GitHub's own Camo image proxy; GitHub fixed it by 14 August 2025, and it was not an in-the-wild leak. OWASP names the CI/CD class Poisoned Pipeline Execution (CICD-SEC-4). Bubblewrap's security model is caller-defined sandbox policy, and this gate is not a SLSA implementation or certification. None of these reports' numbers are measurements from our fixtures. The New York date is the Zenity AI Agent Security Summit at Pier Sixty on Wednesday 21 October 2026. The fifteen minutes are the session brief.

10 Implications

A test is evidence because it is independent of what it tests. When the same agent writes the code and the test, the test's green result is a report on the agent's own work.

Protecting the checker executable did not protect the expected values it used. The vulnerable configuration let the worker change those values and obtain a passing result for unchanged code.

Moving the authoritative expected values to the controller removed that route in the recorded experiment.

Three of the four doors close by taking a permission away: a task-scoped key instead of a broad one, a read-only mount instead of a writable one, a broker with one fixed destination instead of a route. The fourth cannot close that way. Writing tests is part of the job, so locking the answer file would remove the work along with the risk. The controller keeps a second copy of the expected values instead. The worker's copy is a proposal; the controller's copy decides.

An agent can still propose tests. Changes to those tests need independent approval before they become release criteria. The same review should examine the scope of credentials, access to shared files, and the operations available through outbound services. Negative tests should be paired with an intended operation that still succeeds under matching conditions. A no only counts if the yes still works: a host that can do nothing passes every deny-only test, and so did our first release test, which refused substitute bytes only after the approval was already spent.